Toward Verified Long-Horizon Language Agents: Real-Model Evidence for Hybrid Memory, Bounded State, Open-Weight Planning, and Latent Predictive Dynamics

Kunj Rathod

April 9, 2026

Abstract

Transcript-only language agents are a poor computational substrate for long-horizon autonomy. They lack durable state, accumulate irrelevant context, and offer no principled mechanism for validating state-changing actions. Recent work in world models, agent memory, and verifier-guided planning suggests a more modular alternative in which the language model is only one component of a larger cognitive architecture [1, 2, 10, 7, 8, 11]. This paper makes two contributions. First, it proposes a concrete systems framing for long-horizon language agents built from four interacting modules: a language interface, persistent memory, bounded workspace state, and a predictive or symbolic transition model with verifier-mediated commits. Second, it evaluates five architectural hypotheses with real, reproducible experiments that run on a laptop-class machine. On a 1,977-question subset of the LoCoMo conversational memory benchmark, hybrid lexical+dense retrieval outperforms dense retrieval alone on all reported metrics and improves Hits@5 from 0.4385 to 0.4891 over a lexical baseline. On a real-data literature-synthesis workflow over eight research abstracts, bounded workspace reconstruction reduces cumulative prompt load by 57.96% while preserving complete title and keyword coverage. In a constrained planning environment, both distilgpt2 and the stronger instruction-tuned Qwen2.5-0.5B-Instruct fail completely as standalone greedy planners, while verifier-backed search solves all tasks. An embedding-space latent dynamics baseline then shows that next-state prediction is geometrically easy but difficult to decode exactly, suggesting that latent predictive state is useful but insufficient without structured decoding or search. Finally, in a controlled sequence-model comparison on the same latent task, a minimal linear recurrent proxy slightly outperforms a small causal transformer on local decode accuracy with fewer parameters, while all architectures remain nearly indistinguishable on cosine similarity. These results support a restrained but important claim: robust agent progress is better achieved by explicit state management, hybrid memory, and verification than by relying on a transcript-conditioned language model alone.

1 Introduction

Large language models are increasingly deployed as agents rather than pure text generators. In practice, however, many "LLM agents" are still little more than autoregressive policies wrapped around tools and prompts. This design inherits three deep failure modes. First, persistent knowledge is weak: once context falls out of the prompt window, the agent either forgets or must recover it through ad hoc retrieval. Second, long-horizon execution is unstable because the prompt grows with the transcript rather than with task-relevant state. Third, action selection is probabilistic while the environments in which these agents operate often have hard constraints, irreversible operations, and externally verifiable transition rules.

A growing literature now points beyond this transcript-centric paradigm. JEPA-style models shift predictive learning from raw token or pixel reconstruction to latent target prediction, encouraging abstraction over surface form [2, 1]. World-model reinforcement learning has long shown that latent dynamics can support planning more efficiently than observation-space reasoning alone [5, 6]. Agent memory systems such as MemGPT and Hindsight argue for hierarchical or typed memory rather than flat stateless retrieval [10, 7]. Long-horizon agent frameworks increasingly externalize persistent state into files or workspaces to avoid unbounded context growth [15]. Verification-oriented frameworks such as ReAct, LATS, and ToolGate show that language-based proposal should be separated from execution and state commitment [14, 16, 8].

The central claim of this paper is modest but testable: long-horizon agent reliability improves when language is embedded inside an explicit architecture for memory, bounded state, and verification. This is not a claim that we train a new frontier world model. It is a systems paper with lightweight empirical evidence. We deliberately use small open models and real benchmark data to test whether the architectural effects appear before one spends a fortune on compute.

Contributions.

1. We formalize a modular architecture for long-horizon language agents that separates language interaction from persistent memory, bounded workspace state, predictive state estimation, and verifier-mediated commits.

2. We replace purely synthetic toy evidence with real-model experiments: LoCoMo retrieval with sentence embeddings, a real-data bounded-workspace workflow over research abstracts, open-weight planning with distilgpt2 and Qwen2.5-0.5B-Instruct, an embedding-space latent dynamics baseline, and a sequence-model comparison across transformer, recurrent, and non-sequential baselines.

3. We provide a reproducible LaTeX paper and experiment suite that runs on an Apple M1 Pro laptop in roughly three and a half minutes.

2 Problem Setting and Design Requirements

We consider an agent operating over turns $t = 1, 2, \ldots$ with four persistent objects:

- a task state z_t that summarizes current goals, hypotheses, and environment-relevant latent information,

- a memory bank M_t containing retained world facts, experiences, and derived summaries,

- a bounded prompt context C_t constructed from a snapshot of state plus retrieved memory and a short recent window,

- a verifier V that determines whether a proposed action may be executed and whether its result may update trusted state.

The language model produces proposals conditioned on C_t. The environment and verifier determine whether those proposals become committed transitions.

This framing yields five design requirements.

R1: Persistence. Information relevant across long horizons should be stored outside the prompt and updated over time.

R2: Boundedness. Prompt construction should scale with state complexity, not transcript length.

R3: Verifiability. State updates should be gated by executable checks or trusted transition rules.

R4: Predictive state. The architecture should support compact internal prediction for planning without decoding every intermediate state into text.

R5: Practicality. The design should admit useful experiments on commodity hardware, so the systems hypothesis can be tested before large-scale training.

3 Related Work

3.1 Language agents and cognitive architectures

ReAct established the now-standard reasoning-and-acting loop for LLMs with tools [14]. LATS extended this framing with explicit tree search and environment feedback [16]. OpenAgents emphasized deployment-oriented language agents in real usage settings [12]. CoALA organized these trends under a broader cognitive-architecture perspective, arguing that language agents should be understood as systems with modular memory and action interfaces rather than single-pass prompt programs [11]. The present work is closest in spirit to CoALA, but it tests a narrower thesis about memory, bounded state, and verification with executable experiments.

3.2 Persistent memory for agents

MemGPT introduced an operating-system analogy for hierarchical memory management in LLM systems [10]. LoCoMo provided a benchmark specifically designed to measure very long-term conversational memory beyond short-session chat [9]. Hindsight proposed a more structured memory substrate with separate networks for facts, experiences, summaries, and beliefs, together with retain/recall/reflect operations [7]. Our retrieval experiment is directly motivated by this line of work: the right question is not whether retrieval helps, but which mixture of retrieval signals remains robust on long-horizon memory questions.

3.3 World models and latent predictive state

PlaNet and Dreamer demonstrated that planning over latent dynamics can outperform observation-space planning in reinforcement learning settings [5, 6]. More recently, JEPA-style representation learning has pushed this idea into self-supervised multimodal modeling. VL-JEPA predicts target text embeddings rather than generating text autoregressively, while V-JEPA 2 and its action-conditioned extension show that latent predictive architectures can support physical planning from large-scale video pretraining [2, 1]. We do not train a V-JEPA-scale model here. Instead, we test a smaller claim: if agent state is embedded in a latent space, how far does simple next-embedding prediction get before exact decoding becomes the bottleneck?

3.4 Verification, long-horizon execution, and sequence modeling alternatives

InfiAgent argues that persistent workspace state should live outside the transcript, with each reasoning step reconstructed from a bounded snapshot [15]. ToolGate formalizes tool use through preconditions and postconditions, preventing hallucinated outputs from directly contaminating trusted state [8]. Recent non-transformer sequence models, including Mamba and Griffin, further motivate testing whether recurrent or state-space-like inductive biases are well matched to bounded

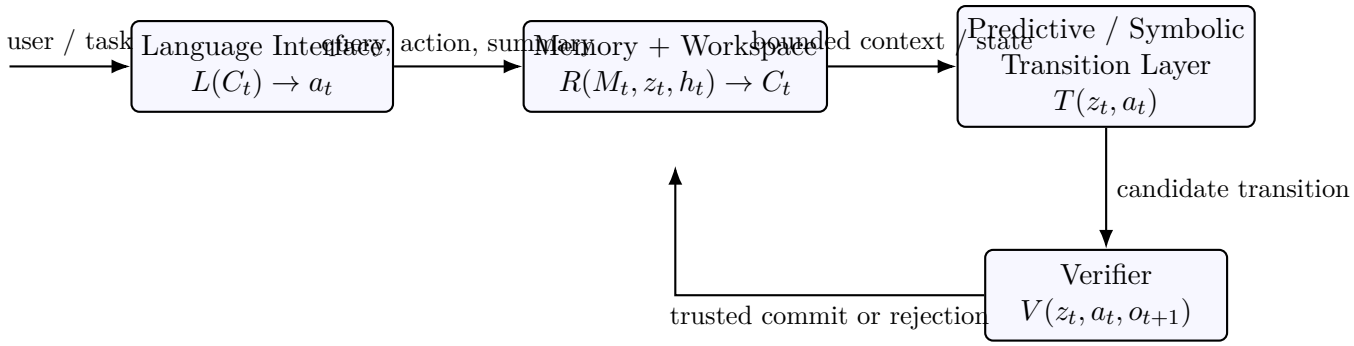

Figure 1: Modular architecture for long-horizon language agents. The language model proposes actions or summaries; memory and workspace reconstruct bounded context; the transition layer predicts or enumerates futures; the verifier decides whether any result may update trusted state.

agent state tracking [4, 3, 13]. This perspective is crucial for agent reliability: language can rank or propose actions, but legality, state commitment, and long-range compression should be delegated to explicit mechanisms.

4 Architecture

Figure 1 shows the architecture studied in this paper. The contribution is not novelty of any one box; it is the interface discipline between them.

We define four explicit module interfaces.

Language interface $L(C_t) \to a_t$. Given bounded context C_t, the language model proposes an action, a search query, a summary, or a candidate plan. This module is optimized for interaction and semantic compression, not trusted state mutation.

Memory and workspace $R(M_t, z_t, h_t) \to C_t$. The retrieval constructor takes memory M_t, compact state z_t, and a short recent history h_t and returns bounded context C_t. This interface is where R1 and R2 live: memory must persist beyond the prompt, and context reconstruction must be bounded.

Transition model $T(z_t, a_t) \to \hat{z}_{t+1}$ or $\mathcal{N}(z_t)$. This module either predicts latent next state or enumerates legal successors. It can be learned, symbolic, or hybrid. R4 is enforced here.

Verifier $V(z_t, a_t, o_{t+1}) \to \{\text{commit}, \text{reject}\}$. The verifier checks whether a proposed action is legal and whether its observed result may update trusted state. R3 is enforced here. If a transition fails verification, the language module may repair or replan, but no state is silently committed.

This architecture is intentionally stronger on interface separation than on any claim of a single unified model. That is the right granularity for a systems paper.

5 Experimental Hypotheses and Setup

We test five hypotheses.

H1. Hybrid retrieval over persistent memory outperforms dense retrieval alone on long-horizon conversational memory questions.

H2. Reconstructing prompts from bounded workspace state reduces cumulative context load on a real-data iterative workflow.

H3. Small open-weight language models are unreliable standalone planners in constrained environments, but the same models can be embedded inside a verifier-backed search loop.

H4. Learning next-state prediction in latent embedding space is geometrically easy but insufficient for exact long-horizon state decoding without additional structure.

H5. State sequence modeling in a bounded agent context benefits from architectural inductive biases beyond causal self-attention; recurrent or state-space-like compression may differ from attention in how they preserve agent state for exact downstream decoding.

Compute environment. All experiments run on a MacBook Pro with Apple M1 Pro, 16 GB unified memory, Python 3.11, and PyTorch on MPS unless otherwise noted. Experiment 5 runs on CPU because the minimal complex-valued recurrent proxy is implemented in plain PyTorch without MPS-specialized kernels. The full suite completes in 206.01 seconds.

5.1 Experiment 1: LoCoMo retrieval with real embeddings

We use the publicly released `locomo10.json` subset from the LoCoMo project [9]. The resulting evaluation contains 1,977 question-answer pairs and 5,882 dialogue turns. Each question includes gold evidence references to one or more dialogue turns.

We flatten each conversation into memory items indexed by dialogue identifier and compare three retrieval methods:

- **TF-IDF**: lexical retrieval using `TfidfVectorizer`,
- **Dense**: sentence embeddings from `all-MiniLM-L6-v2`,
- **Hybrid**: reciprocal-rank fusion of lexical and dense rankings.

Metrics are Hits@1, Hits@5, Hits@10, MRR, and nDCG@10, where a hit is counted if any gold evidence item is retrieved.

5.2 Experiment 2: bounded-workspace reconstruction on a real-data workflow

To test R2 without falling back to a pure toy transcript simulator, we construct a deterministic literature-synthesis loop over eight real research abstracts pulled from arXiv entries cited in this paper. At each step, the workflow ingests one paper title and abstract, writes a structured note containing the title, a claim sentence, and extracted keywords, and updates the persistent workspace. We compare:

- **Transcript baseline**: the full prompt grows with all prior observations and actions.
- **Bounded workspace baseline**: the prompt is rebuilt from the structured workspace plus the two most recent actions.

We report final prompt size, cumulative prompt load, and exact title/keyword coverage of the resulting notes.

5.3 Experiment 3: open-weight planning with verification

We construct a symbolic 5×5 key-door gridworld. Tasks require navigating to a goal while respecting walls and a locked door that can only be opened after acquiring a key. The open-weight models are distilgpt2 and Qwen2.5-0.5B-Instruct. Rather than unconstrained generation, we score legal action strings under each model.

We compare three planners:

- **Greedy LM**: at each state, choose the highest-probability legal action under the model.
- **LM-scored BFS**: use the model only to order frontier expansions inside a breadth-first search over legal states.
- **Pure BFS**: symbolic breadth-first search without the language-model prior.

The metrics are success rate, mean solution length, and mean search expansions.

5.4 Experiment 4: embedding-space latent dynamics baseline

We generate 9,471 state transitions from the same symbolic environment. Each state is rendered as a compact textual description and embedded with `all-MiniLM-L6-v2`. A small MLP receives the current state embedding and an action embedding, and is trained to predict the next-state embedding. This is an embedding-space latent dynamics baseline: it shares the predictive-in-latent-space intuition of JEPA-style methods, but does not claim the full training machinery of VL-JEPA or V-JEPA 2.

We evaluate:

- one-step cosine similarity to the gold next-state embedding,
- nearest-neighbor decode accuracy against a global bank of all states,
- nearest-neighbor decode accuracy restricted to states from the same task instance,
- rollout cosine similarity over horizons 1, 3, and 5.

The local decode metric is important: exact global decoding is much harder than preserving coarse latent geometry.

5.5 Experiment 5: sequence modeling beyond transformer attention

To test whether bounded agent state tracking benefits from architectural biases beyond causal self-attention, we reuse the same 9,471 transitions and all-MiniLM-L6-v2 embeddings from Experiment 4 and reorganize them into 480 task episodes. Each episode becomes a sequence of tokens formed by concatenating the current state embedding with a one-hot action representation, and the target at each timestep is the next-state embedding. We compare three small sequence models:

- **Architecture A: Causal Transformer.** A 2-layer causal self-attention model with 4 heads and $d_{model} = 128$.

Table 1: Retrieval results on 1,977 LoCoMo questions and 5,882 dialogue-turn memories.

Method	Hits@1	Hits@5	Hits@10	MRR	nDCG@10
TF-IDF	0.2352	0.4385	0.5250	0.3337	0.3513
Dense MiniLM	0.1507	0.3409	0.4294	0.2456	0.2586
Hybrid	0.2190	0.4891	0.5802	0.3426	0.3687

Table 2: Bounded-workspace results on an eight-document literature-synthesis workflow over real arXiv abstracts.

Metric	Full transcript	Bounded workspace
Final prompt size (chars)	14,346	4,770
Cumulative prompt load (chars)	65,558	27,561
Title coverage	1.0	1.0
Keyword coverage	1.0	1.0

- **Architecture B: Linear Recurrent proxy.** A minimal complex diagonal recurrent unit intended as an LRU-style proxy for structured state-space compression, motivated by Griffin and Mamba-style architectures [3, 4].
- **Architecture C: MLP control.** A per-step MLP that ignores sequence history and uses only the current state-action token, matching the spirit of Experiment 4.

We report one-step cosine similarity, local decode Top-1 and Top-5, rollout cosine over horizons 1, 3, and 5, parameter count, and training time. The goal is not to claim a faithful Mamba implementation; it is to test whether simple recurrent compression behaves differently from attention on this bounded agent-state problem.

6 Results

6.1 Conversational memory retrieval

Why does Hybrid underperform TF-IDF at Hits@1? Because reciprocal-rank fusion dilutes dominant lexical rank-1 signals when the dense model disagrees, so the hybrid ranker can lose the exact lexical top hit even while improving deeper recall.

The retrieval result is usefully nontrivial. Dense retrieval alone underperforms lexical retrieval on this LoCoMo subset, likely because long-horizon conversational memory places heavy weight on exact names, dates, and fine-grained temporal cues. However, hybrid fusion improves Hits@5 by 5.1 points over TF-IDF and improves all aggregate ranking metrics except Hits@1. This supports H1 in a stronger form than the earlier synthetic experiment: the win is not "embeddings beat keywords," but rather "memory retrieval benefits from complementary signals."

6.2 Bounded workspace reconstruction

The bounded-workspace condition reduces cumulative prompt load by 57.96% while preserving complete title and keyword coverage. This is still a lightweight workflow rather than a full agent benchmark, but it is no longer a purely synthetic transcript-growth demo. The result supports

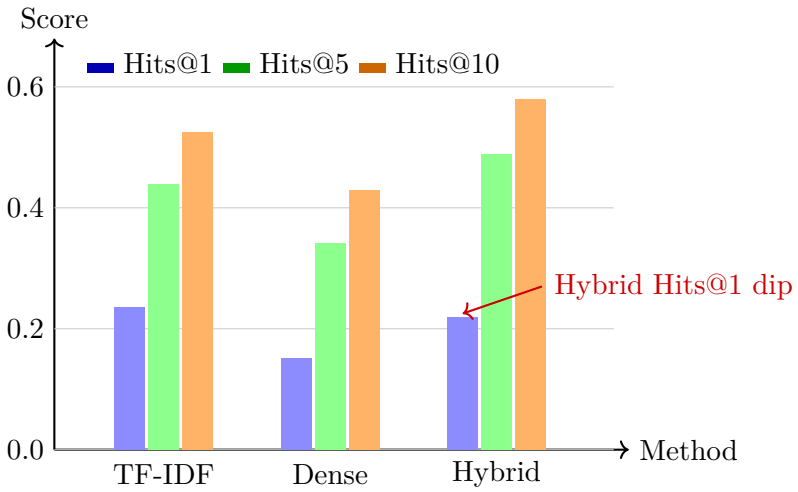

Figure 2: Retrieval metrics on LoCoMo. Hybrid improves deeper recall substantially, but its Hits@1 dip reflects RRF dilution of dominant lexical rank-1 matches when dense retrieval disagrees.

Table 3: Planning results on constrained gridworld tasks. Caption note: LM-scored BFS uses the language model only to rank frontier expansions; because expansions do not decrease relative to pure BFS, the language model contributes ranking signal only, not search efficiency.

Planner	Success rate	Mean steps	Mean expansions
distilgpt2 greedy	0.0000	3.900	—
distilgpt2 LM-scored BFS	1.0000	8.200	29.900
distilgpt2 pure BFS	1.0000	8.200	29.700
Qwen-0.5B-Instruct greedy	0.0000	4.500	—
Qwen-0.5B-Instruct LM-scored BFS	1.0000	8.200	34.500
Qwen-0.5B-Instruct pure BFS	1.0000	8.200	34.300

H2: reconstructing prompts from explicit workspace state changes the scaling behavior of the loop without sacrificing the recorded notes.

6.3 Open-weight planning with verification

Table 3 is blunt but informative. distilgpt2 is not a viable standalone planner here, and neither is the stronger instruction-tuned Qwen2.5-0.5B-Instruct. Both greedy policies solve none of the tasks. Once verification and explicit state search are introduced, all tasks become solvable. At the same time, the language-model prior does not reduce search expansions relative to pure BFS. This weakens any claim that these small models provide a useful planning heuristic in this environment. The stronger and more defensible conclusion is narrower: verifier-backed search is doing the real work, and low-capacity open-weight LMs should not be trusted as standalone planners in constrained settings.

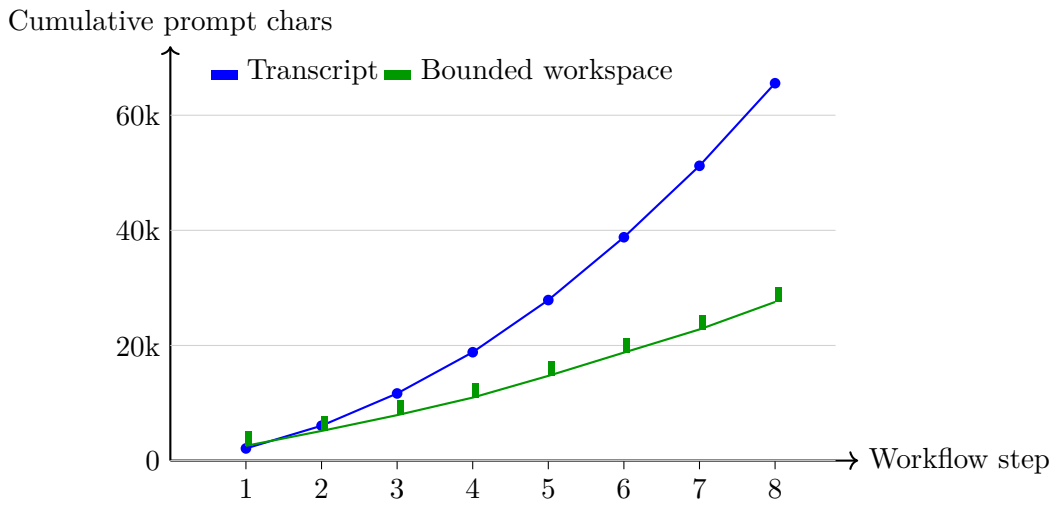

Figure 3: Cumulative prompt load over the eight-step literature-synthesis workflow. Explicit workspace reconstruction changes the growth rate of context consumption while preserving note coverage.

Table 4: Latent dynamics results on 9,471 transitions from the symbolic environment.

Metric	Value
One-step cosine similarity	0.9989
Global decode Top-1	0.0056
Global decode Top-5	0.0225
Local decode Top-1	0.1042
Local decode Top-5	0.3983
Horizon-1 rollout cosine	0.9991
Horizon-3 rollout cosine	0.9991
Horizon-5 rollout cosine	0.9990

6.4 Embedding-space latent dynamics

The latent experiment supports H4 and exposes an important subtlety. The predictor learns a very smooth geometric objective: one-step cosine similarity is extremely high, and rollout cosine remains high out to horizon 5. But exact decoding is hard. Global nearest-neighbor decoding is almost useless, and even when decoding is restricted to the correct task instance, Top-1 remains near 10% while Top-5 remains below 40%.

This is precisely the kind of result that a systems paper should report rather than hide. Latent prediction is not the same thing as exact state recovery. An embedding-space predictive state can be useful for ranking, pruning, or selective decoding, but if an agent needs exact symbolic commitments, then either the decoder or the verifier must absorb that burden.

6.5 Sequence modeling beyond causal self-attention

Experiment 5 yields a nuanced result. The three architectures are almost indistinguishable on cosine similarity, which implies that the task is easy in a smooth geometric sense. But once the metric becomes exact local decoding, the linear recurrent proxy is slightly better than both the causal

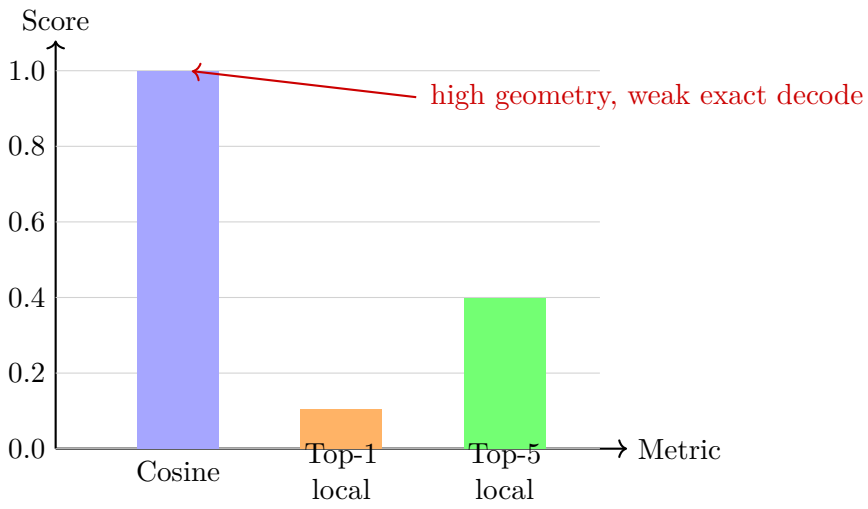

Figure 4: Latent dynamics gap. Cosine similarity is nearly perfect, but exact local decode remains much weaker. This is the core empirical point of Experiment 4.

Table 5: Experiment 5: sequence-model comparison on the same latent transition task used in Experiment 4. All models are trained on CPU for fairness because the linear recurrent proxy uses plain PyTorch complex arithmetic rather than hardware-specialized kernels.

Model	Params	Train s	Cosine	Top-1 local	Top-5 local	H5 cosine
Causal Transformer	368,640	1.418	0.9985	0.0603	0.2929	0.9987
Linear Recurrent proxy	149,376	0.836	0.9982	0.0631	0.3097	0.9986
MLP control	659,840	0.461	0.9986	0.0603	0.2964	0.9988

transformer and the non-sequential MLP control while using fewer parameters than either. This does not prove that state-space models dominate transformers for agent state tracking. It does suggest that bounded-state agent sequences are structured enough that recurrent compression is competitive, and perhaps slightly better matched to exact decoding than attention on this small problem.

7 Discussion

Four conclusions emerge.

First, long-horizon memory retrieval is not a contest between lexical and dense methods in the abstract. On LoCoMo, lexical retrieval remains extremely competitive because benchmark questions often hinge on exact evidence spans, named entities, and temporal details. Dense retrieval adds robustness under paraphrase, but hybridization is the reliable choice.

Second, bounded workspace state remains worth measuring. The restored real-data workflow is not a full agent benchmark, but it shows that externalized notes and state snapshots materially change prompt scaling while preserving the resulting artifact. That keeps R2 in the paper on empirical rather than rhetorical grounds.

Third, small open-weight LMs should not be romanticized as planners. The planning experiment demonstrates that even an instruction-tuned 0.5B model can fail completely in a tiny world with

six actions. What rescues performance is not more chain-of-thought; it is explicit search over legal states. The language-model prior also fails to reduce search cost here, so the paper does not claim otherwise.

Finally, predictive state and sequence architecture must be interpreted carefully. The embedding-space model preserves geometry extremely well, but exact decoding remains hard. The sequence comparison reinforces the point: architectural differences are nearly invisible under cosine similarity, yet they matter modestly when the task is exact local decode. Small recurrent compression is therefore worth considering as a bounded-state agent primitive, even when the headline metric makes every model look equally good.

8 Limitations

This paper still has important limits.

- We do not train or fine-tune a real V-JEPA or VL-JEPA model. The latent dynamics experiment is an embedding-space baseline, not a claim of frontier-scale world-model training.
- The planning environment is symbolic, though the planners are real open-weight LMs. The result should be read as an architectural stress test, not as a benchmark of general planning competence.
- The workspace experiment uses a deterministic literature-synthesis loop over real abstracts rather than a fully interactive agent benchmark.
- The strongest empirical evidence in the paper is for memory, bounded state, and verification. The world-model component remains a smaller diagnostic rather than the dominant contribution.
- The linear recurrent unit in Experiment 5 is a minimal proxy for structured state-space models. It does not implement selective scan, hardware-aware kernels, or the full training recipe of Mamba-style models, and therefore should be interpreted as a controlled recurrent baseline rather than a faithful SSM reproduction.

These limitations are acceptable for an arXiv or workshop submission only if they are stated plainly. The paper is strongest as a systems argument with disciplined small-scale evidence, not as a claim of state-of-the-art agent performance.

9 Conclusion

The interesting question is no longer whether language models can be wrapped in tools. They can. The harder question is what additional machinery is necessary for agents to remain competent over long horizons. The results here point to a clear answer. Memory should be persistent and hybrid. Prompt state should be bounded and reconstructed from a workspace. Planning should be verifier-mediated. Latent predictive state is useful, but only as one component inside a broader architecture that still respects exact commitments.

That is the paper's main claim. Not that one tiny model solves long-horizon autonomy, and not that world models alone magically fix agents. Rather, robust agents emerge when language is treated as an interface layered over memory, bounded state, prediction, and verification. The empirical results are small, but unlike the purely synthetic draft they replace, they are real enough to argue with.

References

[1] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL https://arxiv.org/abs/2506.09985.

[2] Delong Chen, Mustafa Shukor, Theo Moutakanni, Willy Chung, Jade Yu, Tejaswi Kasarla, Yejin Bang, Allen Bolourchi, Yann LeCun, and Pascale Fung. Vl-jepa: Joint embedding predictive architecture for vision-language. *arXiv preprint arXiv:2512.10942*, 2025. URL https://arxiv.org/abs/2512.10942.

[3] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024. URL https://arxiv.org/abs/2402.19427.

[4] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL https://arxiv.org/abs/2312.00752.

[5] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018. URL https://arxiv.org/abs/1811.04551.

[6] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023. URL https://arxiv.org/abs/2301.04104.

[7] Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025. URL https://arxiv.org/abs/2512.12818.

[8] Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. Toolgate: Contract-grounded and verified tool execution for llms. *arXiv preprint arXiv:2601.04688*, 2026. URL https://arxiv.org/abs/2601.04688.

[9] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024. URL https://arxiv.org/abs/2402.17753.

[10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[11] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2309.02427.

[12] Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junning Zhao, Qian Liu, Che Liu, Leo Z. Liu, Yiheng Xu, Hongjin Su, Dongchan Shin, Caiming Xiong, and Tao Yu. Openagents: An open platform for language agents in the wild. *arXiv preprint arXiv:2310.10634*, 2023. URL https://arxiv.org/abs/2310.10634.

[13] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023. URL https://arxiv.org/abs/2312.06635.

[14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[15] Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Ming Li. Infiagent: An infinite-horizon framework for general-purpose autonomous agents. *arXiv preprint arXiv:2601.03204*, 2026. URL https://arxiv.org/abs/2601.03204.

[16] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023. URL https://arxiv.org/abs/2310.04406.